Accuracy-Preserving Summation Algorithm

Input file:	`standard input`
Output file:	`standard output`
Time limit:	10 seconds
Memory limit:	1024 megabytes

In the classic high-performance computing domain (HPC), the vast majority of computations are conducted in double-precision 64-bit floating-point numbers (fp64, double-precision, IEEE-754 binary64). The rise of Deep Neural Networks (DNNs) resulted in hardware (HW) capable of processing 16-bit floating point numbers (fp16, half precision, IEEE-754 binary16) up to 16 times faster in terms of floating-point operations per second (flops) and up to 4 times faster in terms of memory bandwidth (BW). At the same time, the short mantissa and exponent for fp16 numbers lead to a very fast loss of precision of computations, producing wrong computational results without any option to recover them in reduction operations of size greater than approximately 2000. As the typical problem size in HPC is much larger than 2000, this makes fp16 computations almost useless. To surmount this major roadblock, smarter algorithms for reduction operations are needed.

Description. There is a sequence of floating-point numbers stored in IEEE-754 binary64 (double precision, fp64) format x_i of length N. The sequence needs to be summed up to $S = x_1 + x_2 + \ldots + x_N$. As professional computer equipment with native support for fp16 is usually unavailable to the general audience, we propose to do operations in a simplified simulated environment, that is, we do computations in fp64 format with mantissa and exponent cut to the range admissible in fp16. In particular, small values that do not fit fp16 admissible range turn into zeros, while excessively large values turn into infinities.

Objective. Your objective is to sum up as many sequences as possible as fast as possible and as accurately as possible. Please note that you may do summation in fp64 format, but the summation process would be slow though accurate. If you do plain summation in fp16 format, it can be fast, but inaccurate, especially for larger sequences.

Input

The input consists of a single line. It starts with an integer N representing the number of values in the sequence. The following N double precision numbers form the sequence x_i, where $i = 1, \ldots, N$.

Variable constraints:

- Length of the sequence: $2 \leq N \leq 1\,000\,000$.
- Value of any individual number in the sequence: legal IEEE-754 binary64 value stored in decimal format.

Note that the actual binary64 value is not always exactly equal to the given decimal value. Instead, the actual given value is the closest number representable in binary64. When reading the input, most programming languages will do the conversion for you automatically.

It is guaranteed that every number in the sequence either is 0 or has absolute value between 10^{-300} and 10^{300}, inclusive.

Output

Print a single line which will describe the summation process. The line should contain an encoded algorithm for the summation. We use this encoding to do actual summation and report the result to prevent the need to seek hardware capable of doing fp16 operations natively.

An encoded algorithm consists of the data type to use, followed by a list of values to sum up using this data type. The result of the algorithm is the sum of the given values, as computed in the given data type, from left to right, in the given order. It looks as follows:

`{type:value_1,value_2,...,value_k}`

As you can see, the whole algorithm is surrounded by curly brackets ("`{`" and "`}`"). The next character represents one of the three possible data types:

- "`d`" for fp64 summation,
- "`s`" for fp32 summation,
- "`h`" for fp16 summation.

Then goes a colon ("`:`"). It is followed by a non-empty list of values to sum up, separated by commas ("`,`"). Note that there are no spaces.

Each value can be one of the following:

- an integer from 1 to N indicating a position in the input sequence: in this case, the value comes directly from the input;
- another algorithm: in this case, the value is the result of this algorithm.

Some examples:

- `{d:1,2,3,4}` tells to use double precision to compute $x_1 + x_2 + x_3 + x_4$;
- `{h:4,3,2,1}` tells to use half precision to compute $x_4 + x_3 + x_2 + x_1$;
- `{d:{s:3,4},{h:2,1}}` tells to use double precision to compute $y + z$, where:
  - y is to use single precision to compute $x_3 + x_4$,
  - z is to use half precision to compute $x_2 + x_1$;
- `{h:1,4,{d:3,2}}` tells to use half precision to compute $x_1 + x_4 + y$, where:
  - y is to use double precision to compute $x_3 + x_2$.

Each input value must be used exactly once.

Scoring

In this problem, there are 2 example tests and 76 main tests. Each main test is scored as follows.

The first part of scoring is related to accuracy. We denote the sum calculated by the solution as S_c. Next, we calculate the expected sum S_e as precisely as we can, and store it in binary64 format. Then the accuracy score is calculated as:

$$A = \left(\max \left(\frac{|S_c - S_e|}{\max\left(|S_e|, 10^{-200}\right)}, 10^{-20} \right) \right)^{0.05}.$$

For example, if the calculated sum is 99.0 and the expected sum is 100.0, the accuracy score is $A = \left(\frac{|99-100|}{|100|}\right)^{0.05} = \left(\frac{1}{100}\right)^{0.05} = 0.794328....$ If the relative error is $\frac{1}{1000}$, the accuracy score is $A = \left(\frac{1}{1000}\right)^{0.05} = 0.707945....$ If the solution is exact, we get $A = \left(10^{-20}\right)^{0.05} = 0.1$.

The second part of scoring is related to the types used for summation. We define the weight W of the algorithm as follows. If the algorithm performs k additions (adding up $k + 1$ values), its weight is:

- $1 \cdot k$ for half precision,

- $2 \cdot k$ for single precision,
- $4 \cdot k$ for double precision.

Additionally, if the algorithm contains other algorithms, their weights are computed recursively and added to the weight of the parent algorithm.

The third part of scoring is related to a penalty for memory reads. We list the N numbers appearing in the algorithm from left to right, **omitting all curly brackets**. We divide the numbers into consecutive blocks of 16 elements; the last block may contain less than 16 elements. In each block, its first element i initiates a memory read. For every other element j of the block, if $|j - i| > 15$, it is too far from the memory which was read for this block, and you get a penalty. The penalties increase gradually: the x-th penalty you get will be $x/20\,000$. The penalty counter x is global, it persists across different blocks. All the penalties add up to the total penalty P.

For example, consider the following algorithm: `{s:1,2,3,4,5,6,7,8,9,10,{d:20,19,18,17,16},11,12,13,14,15}`. It performs 15 additions in single precision, and one of its elements is also an algorithm which performs 4 additions in double precision. Thus its weight is $W = 15 \cdot 2 + 4 \cdot 4 = 46$. The first memory read block is `1,2,3,4,5,6,7,8,9,10,20,19,18,17,16,11`, the initial read is at position 1, and the positions in the block that are more than 15 units away are 20, 19, 18, and 17, so we get 4 penalties. The second memory read block is `12,13,14,15`, the initial read is at position 12, and there are no penalties. The total penalty is $P = (1 + 2 + 3 + 4)/20\,000 = 0.0005$.

Putting the second and third part together, we calculate the average cost for a single operation as:

$$C = \frac{W + P}{N - 1}.$$

Then the data score is calculated as:

$$D = \frac{10.0}{\sqrt{C + 0.5}}.$$

Lastly, taking the accuracy score into account, the score for the test is:

$$Score = \frac{D}{A}.$$

All individual test scores for main tests sum up together to produce the final score. Examples are checked, but do not contribute to the score.

Examples

standard input	standard output
2 -4.815473e+04 -1.862622e+04	{d:1,2}
3 -4.815473e+01 1.862622e+02 2.997603e+02	{d:1,2,3}

Note

Two sets of tests are prepared in this problem: preliminary tests and final tests. For the duration of the competition, each submission is tested on the preliminary set of tests. When the competition is finished, for each contestant:

- The jury takes the **latest** submission with **non-zero** score on preliminary tests.
- This submission is tested on the final set of tests.
- The contestants are ranked according to their performance on the final tests.

The two sets of tests are designed to be similar, but not identical.